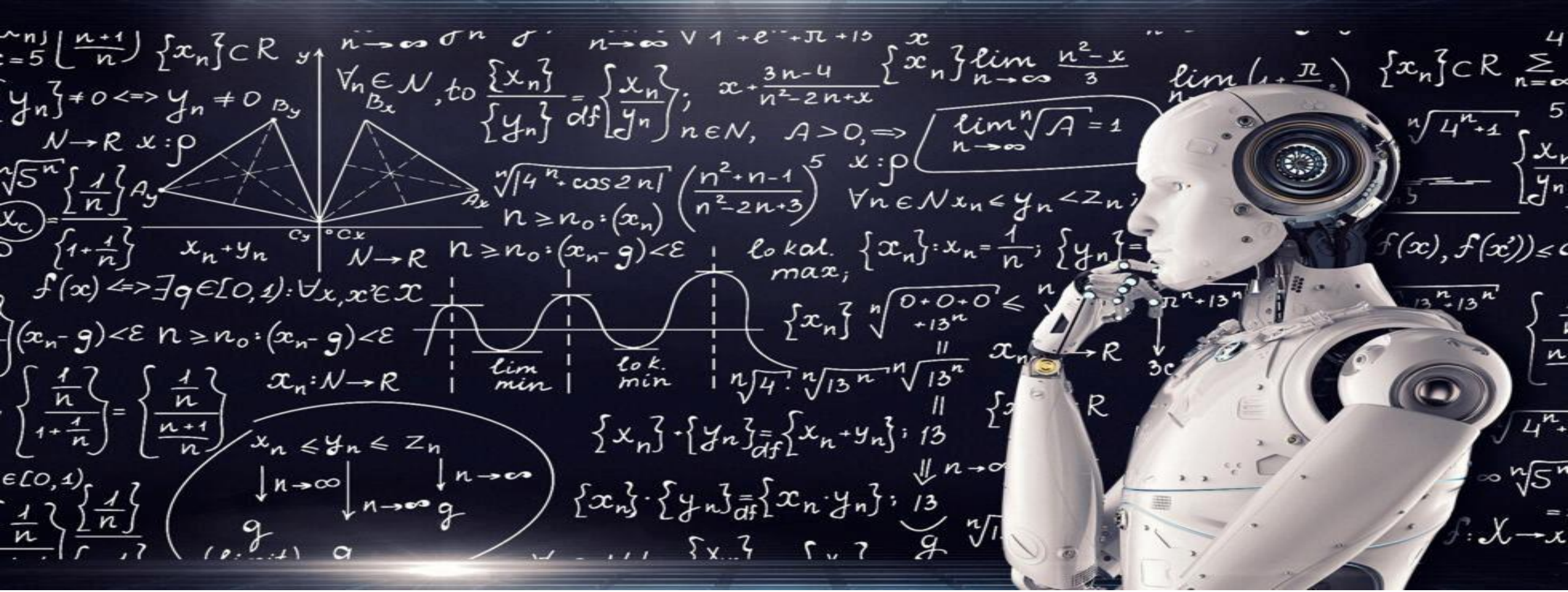


ADVANCED MACHINE LEARNING (ARI 501 - DSC501)

Dr.Ahmed Anter
Associate Professor of Computer Science, CSIT, E-JUST, Egypt
Spring, 2026

Decision Tree
Classification – Regression

DECISION TREE

Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems in the form of a tree structure.

It breaks down a dataset into smaller and smaller subsets while at the same time an associated decision tree is incrementally developed.

The final result is a tree with **decision nodes** and **leaf nodes**.

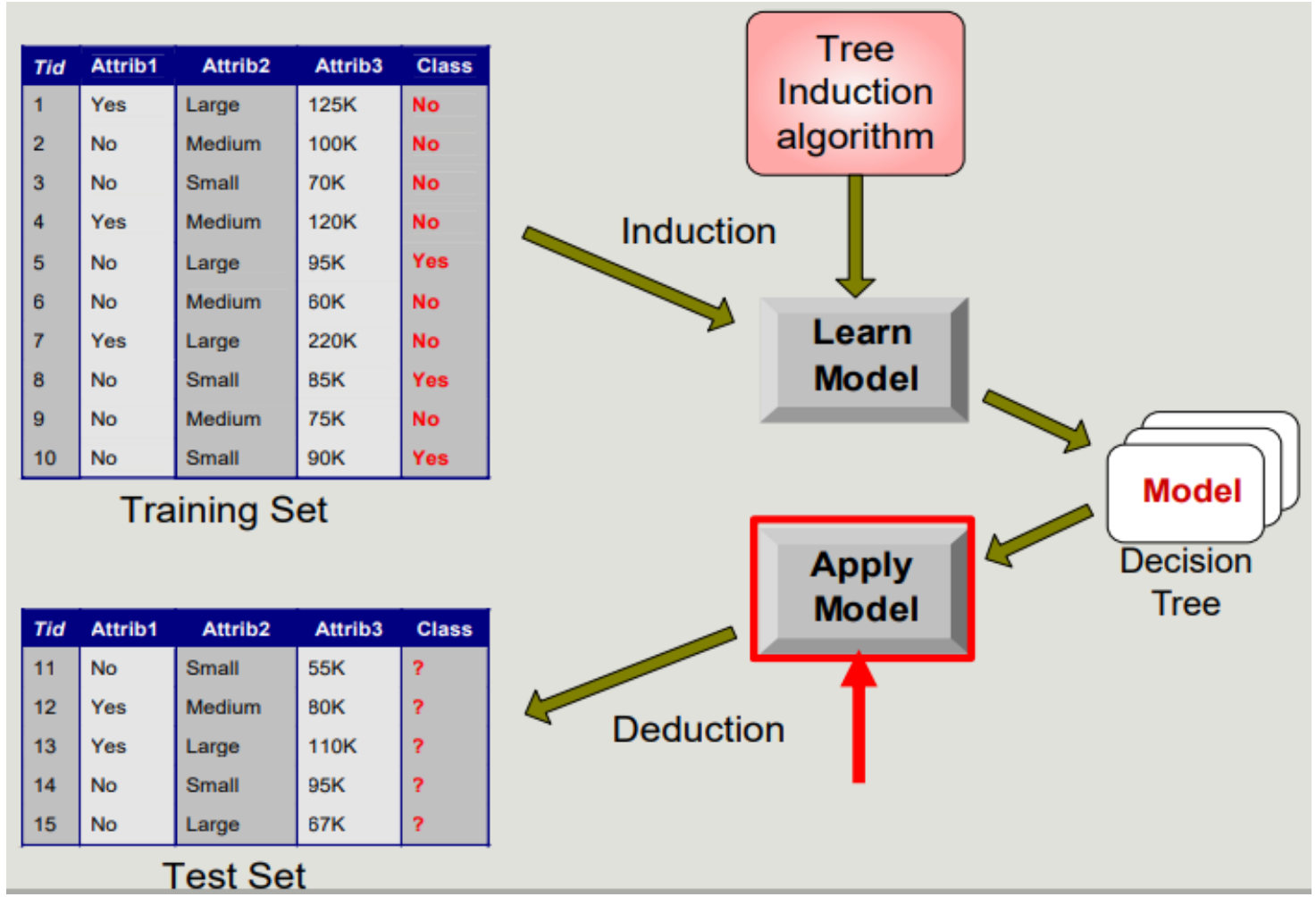
A decision node (e.g., Outlook) has two or more branches (e.g., Sunny, Overcast and Rainy).

Leaf node (e.g., Play) represents a classification or decision.

The topmost decision node in a tree which corresponds to the best predictor called **root node**.

Decision trees can handle both **categorical** and **numerical** data.

APPLYING THE ML MODEL



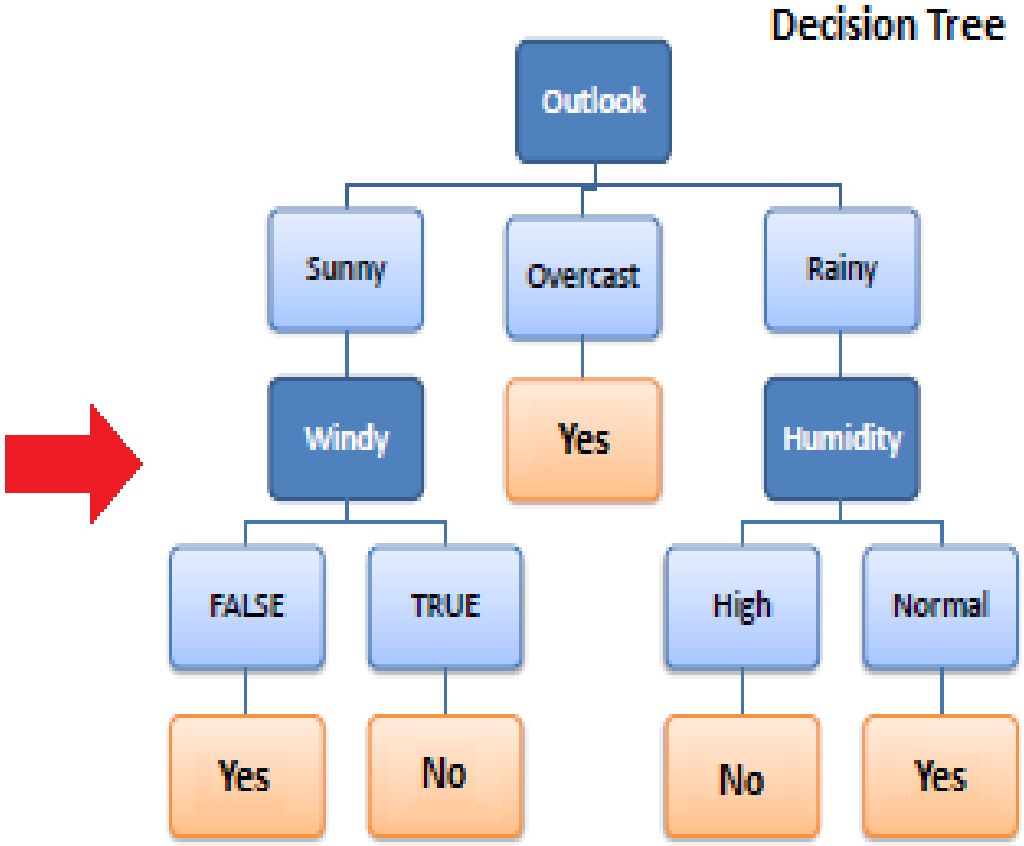
Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

APPLY DECISION TREE MODEL

Predictors | Target

Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



LEARNING THE MODEL

The decision tree number grows exponentially with the number of attributes. Algorithms generally use greedy techniques that locally make the "best" choice.

Finding an optimal decision tree is a NP-Complete problem, but many heuristic algorithms are available and very efficient.

Most approaches run a top-down recursive partition based on greedy criteria.

COMMON DECISION TREE ALGORITHMS

Many decision tree algorithms are available:

ID3: Uses information gain to select splits.
- *Example*: Used for simple categorical datasets.

C4.5: An improvement over ID3; handles continuous features and missing values.
- *Example*: Often used in academic problems and real-world datasets with both categorical and numeric data.

Random Forest: Builds many decision trees and combines their outputs for better accuracy.
- *Example*: Used in Kaggle competitions and real-world applications where prediction accuracy is crucial.

CART (Classification and Regression Trees): Can produce classification or regression trees; uses Gini impurity.
- *Example*: Used in financial forecasting.

Hunt's Algorithm: An early recursive partitioning method.
- *Example*: Forms the basis for many modern tree algorithms.

Sliq, SPRINT: Scalable algorithms for building trees on large datasets.
- *Example*: Used for data mining in big data settings.

DECISION TREE ALGORITHMS

Algorithm	Key Feature	Example Use Case
ID3	Categorical, information gain	Simple classification tasks
C4.5	Handles numeric & missing data	Real-world mixed datasets
Random Forest	Many trees, averages results	High-accuracy prediction
CART	Classification & regression	Finance, health analytics
Hunt's Algorithm	Recursive partitioning	Basis for other algorithms
Sliq, SPRINT	Scalable to large datasets	Big data mining

ALGORITHM

The core algorithm for building decision trees called **ID3** by J. R. Quinlan which employs a top-down, greedy search through the space of possible branches with no backtracking.

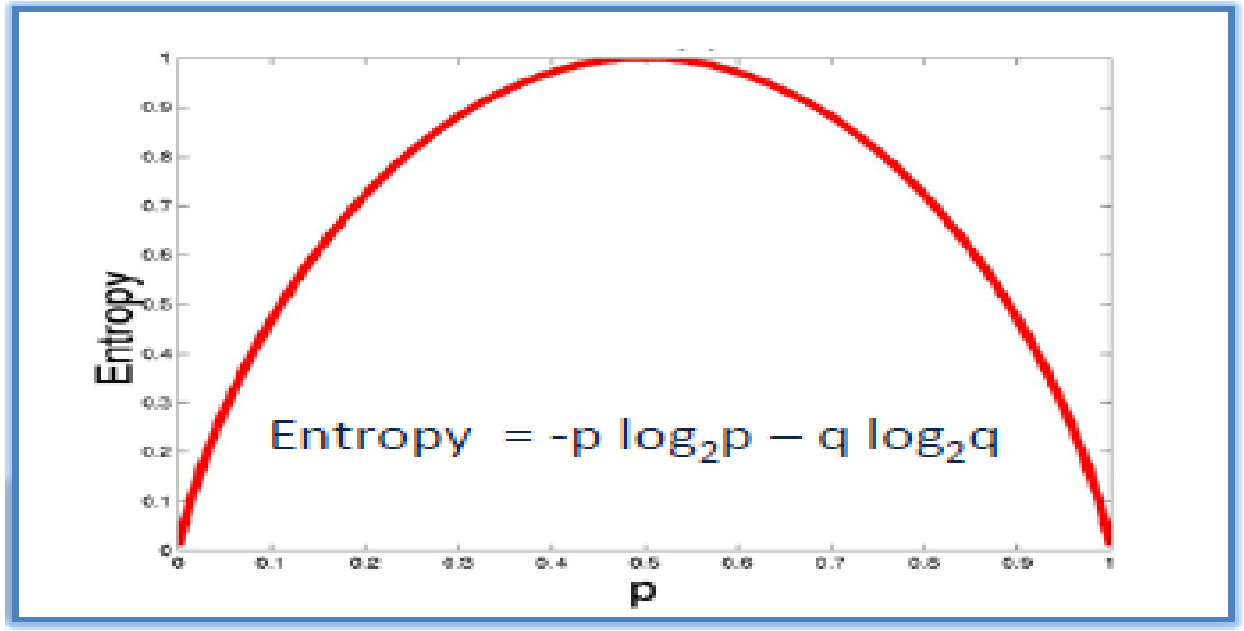
ID3 uses *Entropy* and *Information Gain* to construct a decision tree.

Entropy

- A decision tree is built top-down from a root node and involves partitioning the data into subsets that contain instances with similar values (homogenous).
- ID3 algorithm uses entropy to calculate the homogeneity of a sample. If the sample is completely homogeneous the entropy is **zero** and if the sample is an equally divided it has entropy of **one**.

ALGORITHM

Entropy in the context of decision trees is a measure of **impurity** or **uncertainty** in a dataset. It comes from information theory and helps to quantify the **randomness** or **disorder** in the data with respect to the class labels.



$$\text{Entropy} = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

ENTROPY – FREQUENCY

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Entropy (5,3,2) = Entropy (0.5,0.3,0.2)

= - (0.5 * $\log_2$0.5) - (0.3 * $\log_2$0.3) - (0.2 * $\log_2$0.2)

= 1.49

ALGORITHM
ENTROPY

To build a decision tree, we need to calculate two types of entropy using frequency tables as follows:

a) Entropy using the frequency table of one attribute:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5

Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

ALGORITHM
ENTROPY

b) Entropy using the **frequency table** of two attributes:

$$E(T,X) = \sum_{c \in X} P(c)E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

E(PlayGolf, Outlook) = **P**(Sunny)***E**(3,2) + **P**(Overcast)***E**(4,0) + **P**(Rainy)***E**(2,3)

= (5/14)*0.971 + (4/14)*0.0 + (5/14)*0.971

= 0.693

ALGORITHM

INFORMATION GAIN

The information gain is based on the decrease in entropy after a dataset is split on an attribute.

Constructing a decision tree is all about finding attribute that returns the highest information gain (i.e., the most homogeneous branches).

Step 1: Calculate entropy of the target.

$$
\begin{aligned}
\text{Entropy(PlayGolf)} &= \text{Entropy}(5,9) \\
&= \text{Entropy}(0.36, 0.64) \\
&= -(0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\
&= 0.94
\end{aligned}
$$

ALGORITHM

Step 2: The dataset is then split on the different attributes.

The entropy for each branch is calculated. Then it is added proportionally, to get total entropy for the split.

The resulting entropy is subtracted from the entropy before the split. The result is the Information Gain, or decrease in entropy.

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

ALGORITHM

$$Gain(T, X) = Entropy(T) - Entropy(T, X)$$

G(PlayGolf, Outlook) = **E**(PlayGolf) – **E**(PlayGolf, Outlook)

= 0.940 – 0.693 = 0.247

Step 3: Choose attribute with the largest information gain as the decision node, divide the dataset by its branches and repeat the same process on every branch.

INFORMATION GAIN –THE BEST PREDICTOR?

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

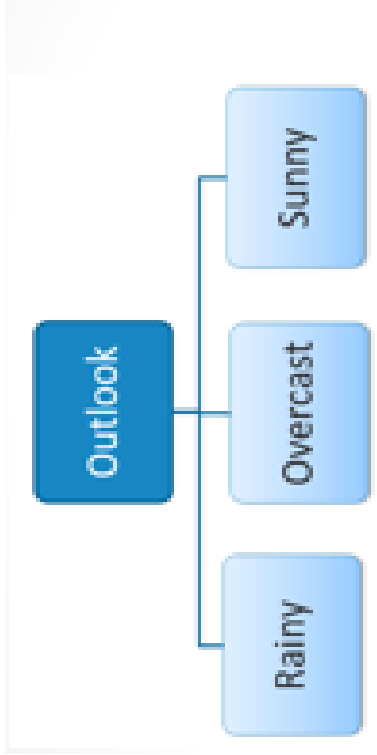
		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

ALGORITHM

★		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

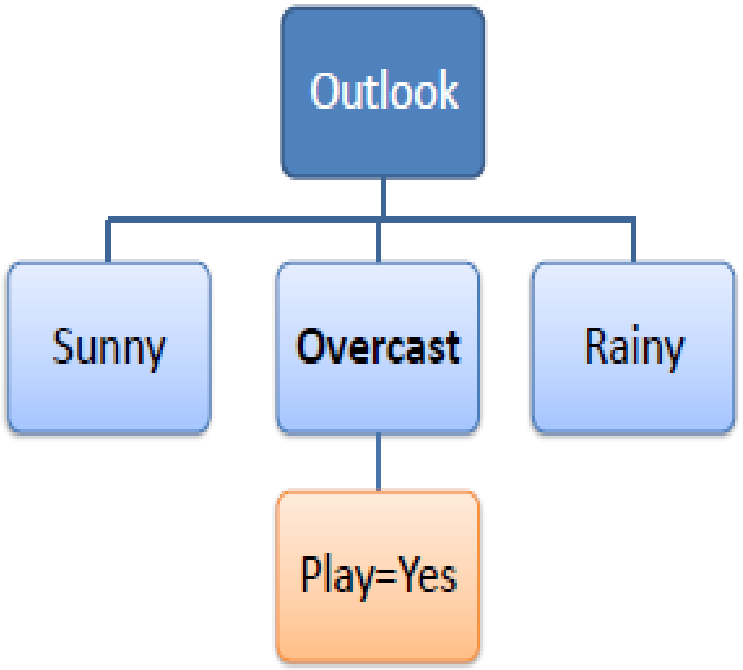


Outlook	Temp	Humidity	Windy	Play Golf
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes
Sunny	Cool	Normal	TRUE	No
Sunny	Mild	Normal	FALSE	Yes
Sunny	Mild	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Overcast	Cool	Normal	TRUE	Yes
Overcast	Mild	High	TRUE	Yes
Overcast	Hot	Normal	FALSE	Yes
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Rainy	Mild	High	FALSE	No
Rainy	Cool	Normal	FALSE	Yes
Rainy	Mild	Normal	TRUE	Yes

ALGORITHM

Step 4a: A branch with entropy of 0 is a leaf node.

Temp.	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes



SUBSET (OUTLOOK = SUNNY)

Temp.	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	Normal	FALSE	Yes
Mild	High	TRUE	No

		Play Golf	
		Yes	No
Temp.	Mild	2	1
	Cool	1	1
Gain = 0.02			

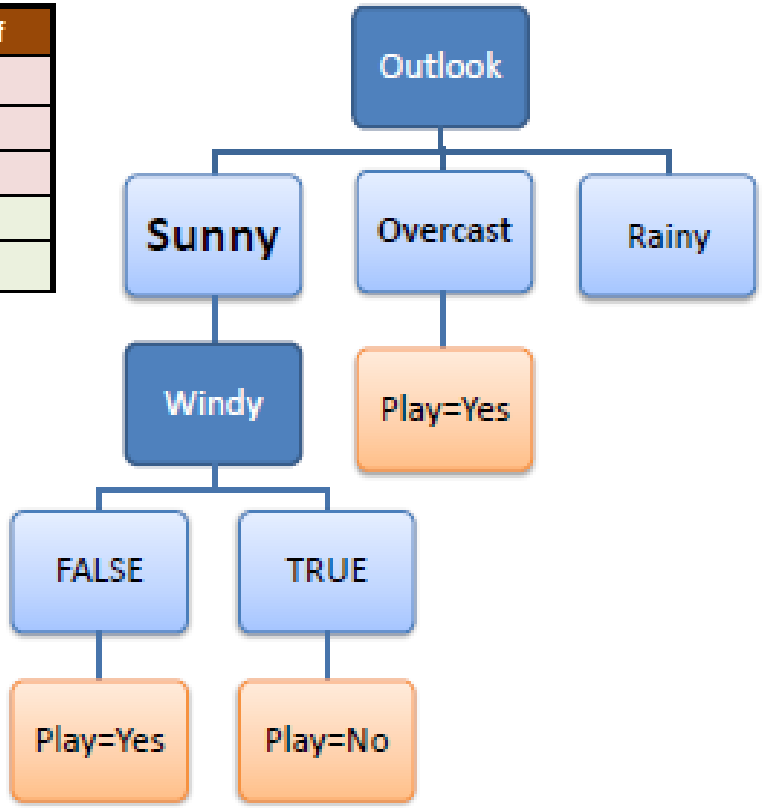
		Play Golf	
		Yes	No
Humidity	High	1	1
	Normal	2	1
Gain = 0.02			

		Play Golf	
		Yes	No
Windy	False	3	0
	True	0	2
Gain = 0.97			

ALGORITHM

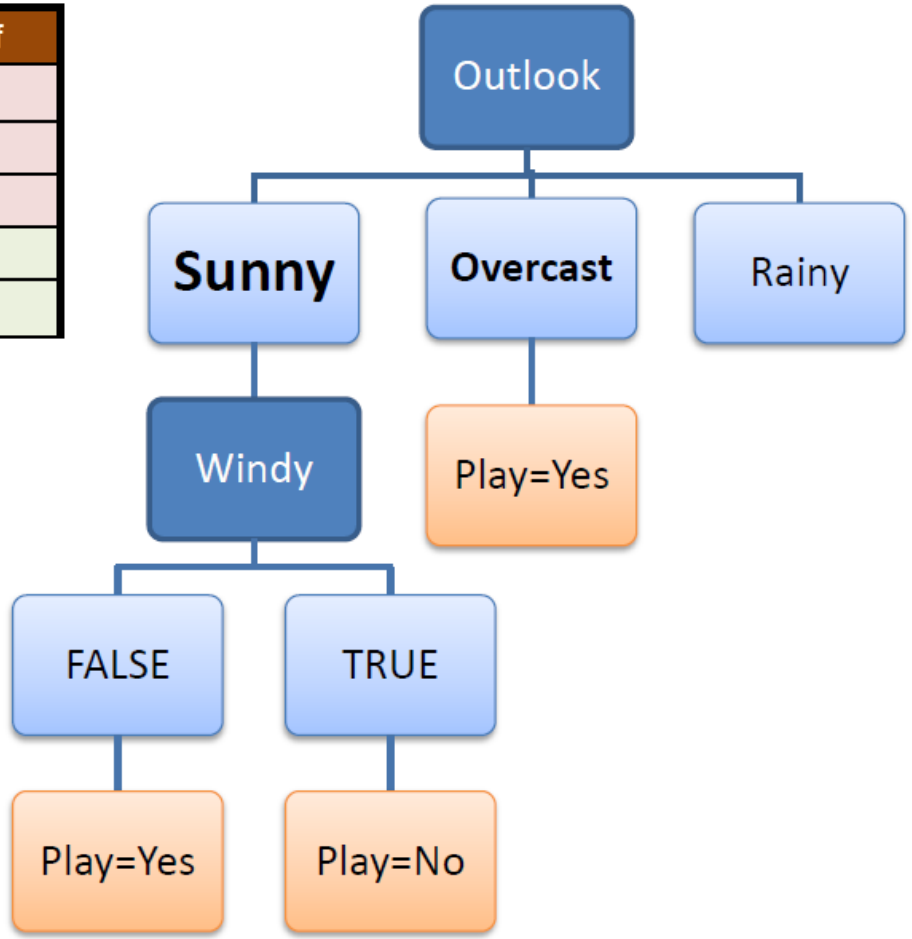
Step 4b: A branch with entropy more than 0 needs further splitting.

Temp.	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



SUBSET (OUTLOOK = SUNNY)

Temp.	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



SUBSET (OUTLOOK = RAINY)

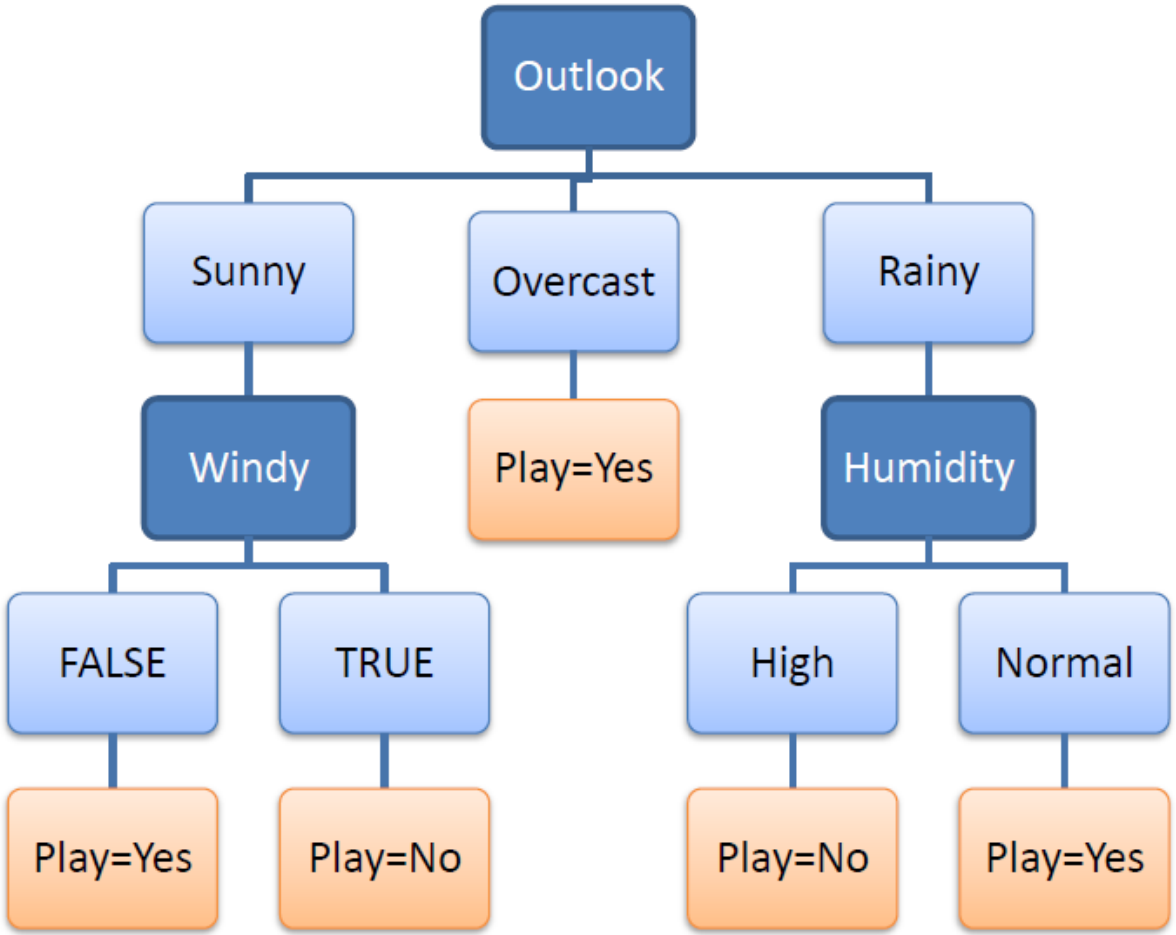
Temp.	Humidity	Windy	Play Golf
Hot	High	FALSE	No
Hot	High	TRUE	No
Mild	High	FALSE	No
Cool	Normal	FALSE	Yes
Mild	Normal	TRUE	Yes

		Play Golf	
		Yes	No
Temp.	Hot	0	2
	Mild	1	1
	Cool	1	0
Gain = 0.57			

		Play Golf	
		Yes	No
Humidity	High	0	3
	Normal	2	0
Gain = 0.97			

		Play Golf	
		Yes	No
Windy	False	1	2
	True	1	1
Gain = 0.02			

SUBSET (OUTLOOK = RAINY)



ALGORITHM

Step 5: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified.

DECISION TREE TO DECISION RULES

A decision tree can easily be transformed to a set of rules by mapping from the root node to the leaf nodes one by one.

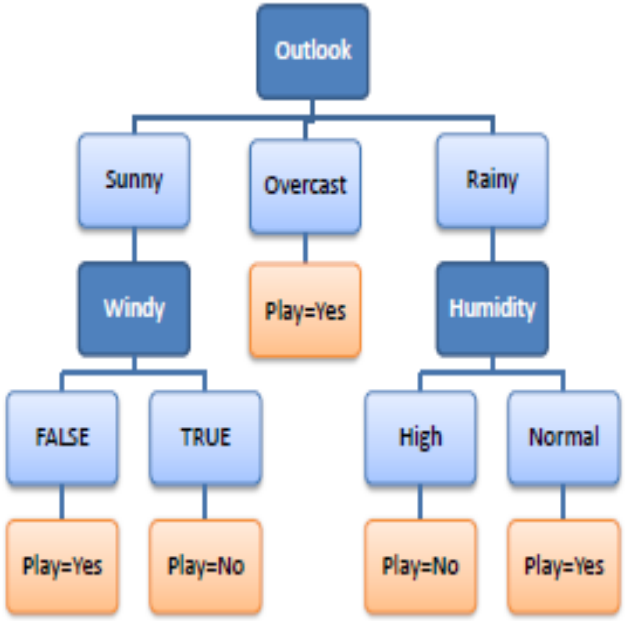
R_1: IF (Outlook=Sunny) AND (Windy=FALSE) THEN Play=Yes

R_2: IF (Outlook=Sunny) AND (Windy=TRUE) THEN Play=No

R_3: IF (Outlook=Overcast) THEN Play=Yes

R_4: IF (Outlook=Rainy) AND (Humidity=High) THEN Play=No

R_5: IF (Outlook=Rain) AND (Humidity=Normal) THEN Play=Yes



WHY USE DECISION TREES?

Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.

The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Working well with the small dataset.

DECISION TREES - ISSUES

Working with continuous attributes (binning)

Overfitting and Pruning

Super Attributes (attributes with many values)

Working with missing values

Attributes with Different Costs

- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.

NUMERIC VARIABLES -BINNING

Temp	B_Temp	Play Golf
85	80-90	No
80	80-90	No
83	80-90	Yes
70	70-80	Yes
68	60-70	Yes
65	60-70	No
64	60-70	Yes
72	70-80	No
69	60-70	Yes
75	70-80	Yes
75	70-80	Yes
72	70-80	Yes
81	80-90	Yes
71	70-80	No

		Play Golf	
		Yes	No
B_Temp	60-70	3	1
	70-80	4	2
	80-90	2	2

NUMERIC VARIABLES AND MISSING VALUES

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	85	High	False	No
Rainy	80	High	True	No
Overcast	?	High	False	Yes
Sunny	70	High	False	Yes
Sunny	68	?	False	Yes
Sunny	65	Normal	True	No
Overcast	64	Normal	True	Yes
Rainy	72	High	?	No
Rainy	69	Normal	False	Yes
Sunny	?	Normal	False	Yes
Rainy	75	Normal	True	Yes
?	72	High	True	Yes
Overcast	81	Normal	False	Yes
Sunny	71	High	True	No

MISSING VALUES

- For the numerical variables replace the missing value with the average or median.

- For the categorical variables replace the missing with.
  - Most common value
  - Most common value at node K

- K Nearest Neighbors (KNN)

DECISION TREE- OVERFITTING

Overfitting is a significant practical difficulty for decision tree models and many other predictive models.

Overfitting happens when the learning algorithm continues to develop hypotheses that reduce training set error at the cost of an increased test set error.

There are several approaches to avoid overfitting in building decision trees

1. Pre-pruning that stop growing the tree earlier, before it perfectly classifies the training set.

2. Post-pruning that allows the tree to perfectly classify the training set, and then post prune the tree.

Practically, the second approach of post-pruning overfit trees is more successful because it is not easy to precisely estimate when to stop growing the tree.

The important step of tree pruning is to define a criterion be used to determine the correct final tree size using one of the following methods:

PRUNING

Pruning: Getting an Optimal Decision tree.

Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset. Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as **Pruning**.

PRUNING STRATEGIES

1. Use a distinct dataset from the training set (called validation set), to evaluate the effect of post-pruning nodes from the tree.

2. Build the tree by using the training set, then apply a statistical test to estimate whether pruning or expanding a particular node is likely to produce an improvement beyond the training set.

- Error estimation
- Significance testing (e.g., Chi-square test) .

3. Minimum Description Length (MDL) principle : Use an explicit measure of the complexity for encoding the training set and the decision tree, stopping growth of the tree when this encoding size (size(tree) + size(misclassifications(tree)) is minimized.

PRUNING OPERATIONS

- Subtree Replacement
- Subtree Raising

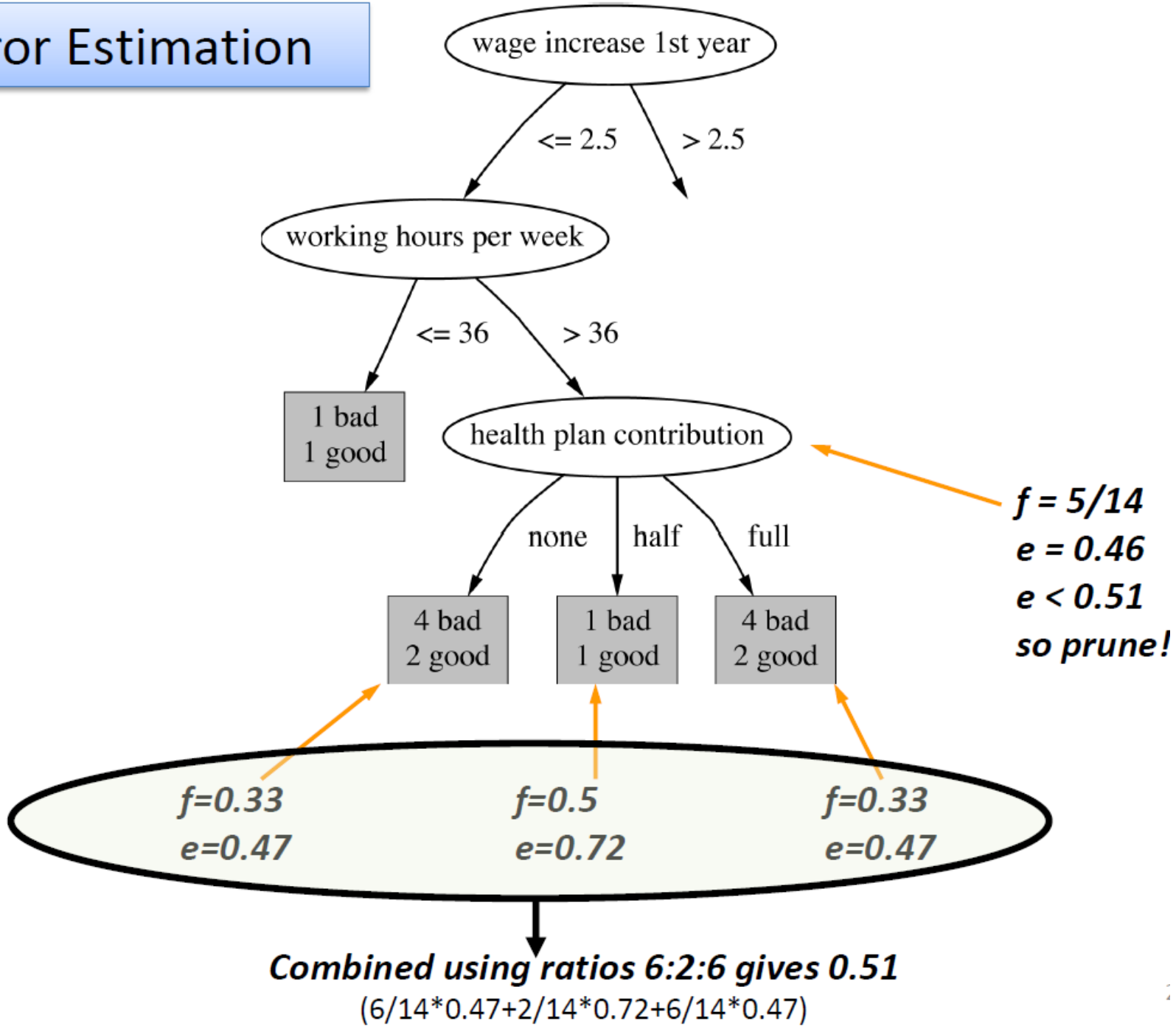
ERROR ESTIMATION

- Error estimate for subtree is weighted sum of error estimates for all its leaves
- Error estimate for a node (upper bound):

$$e = \left(f + \frac{z^2}{2N} + z\sqrt{\frac{f}{N} - \frac{f^2}{N} + \frac{z^2}{4N^2}} \right) \Big/ \left(1 + \frac{z^2}{N} \right)$$

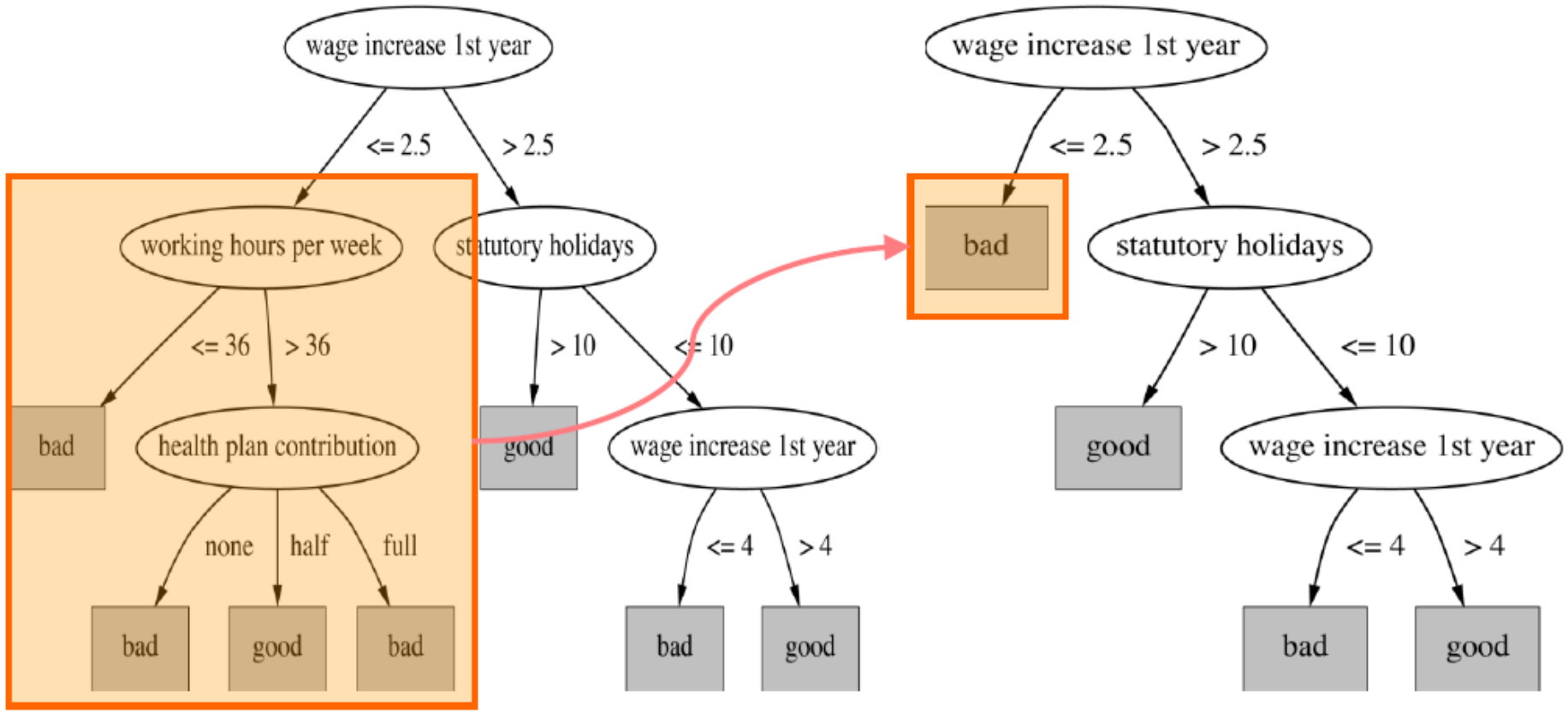
- If c = 25% then z = 0.69 (from normal distribution)
- f is the error on the training data
- N is the number of instances covered by the leaf

Error Estimation



The error rate at the parent node is **0.46** and since the error rate for its children (0.51) increases with the split, we do not want to keep the children.

SUBTREE REPLACEMENT



SUPER ATTRIBUTES

- The information gain equation, G(T,X) is biased toward attributes that have a large number of values over attributes that have a smaller number of values.

- Theses ‘Super Attributes’ will easily be selected as the root, result in a broad tree that classifies perfectly but performs poorly on unseen instances.

- We can penalize attributes with large numbers of values by using an alternative method for attribute selection, referred to as **GainRatio**.

$$\textbf{GainRatio}(T,X) = \textbf{Gain}(T,X) \ / \ \textbf{SplitInformation}(T,X)$$

SUPER ATTRIBUTES

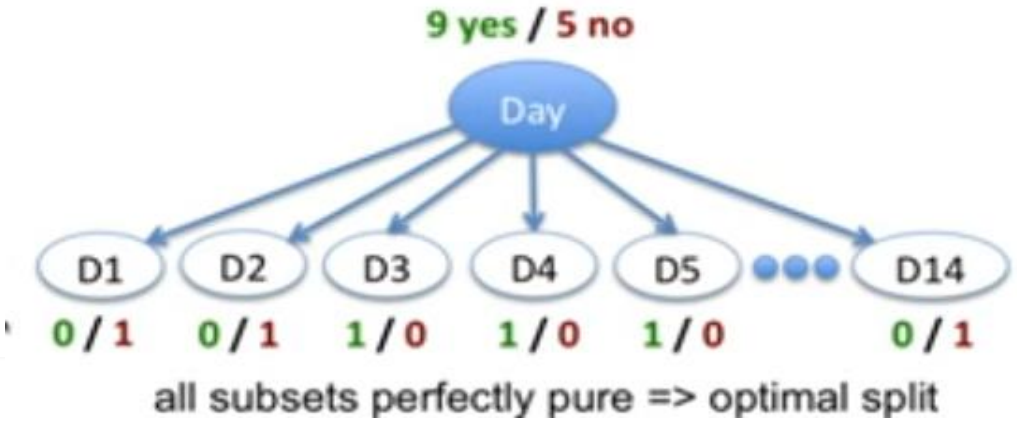
		Play Golf		
		Yes	No	***total***
Outlook	Sunny	**3**	**2**	**5**
	Overcast	**4**	**0**	**4**
	Rainy	**2**	**3**	**5**
Gain = 0.247				

$$Split(T,X) = -\sum_{c \in A} P(c)\log_2 P(c)$$

Split (Play,Outlook) = - (5/14*$\log_2$(5/14) + 4/14*$\log_2$(4/15) + 5/14*$\log_2$(5/14))
= 1.577

Gain Ratio (Play,Outlook) = 0.247/1.577 = **0.156**

SUPER ATTRIBUTES



Example: The following example shows a frequency table between the target (Play Golf) and the ID attribute which has a unique value for each record of the dataset.

		Play Golf		
		Yes	No	*total*
ID	id1	1	0	1
	id2	0	1	1
	id3	1	0	1
	id4	1	0	1
	id5	0	1	1
	id6	0	1	1
	id7	1	0	1
	id8	1	0	1
	id9	0	1	1
	id10	1	0	1
	id11	1	0	1
	id12	0	1	1
	id13	1	0	1
	id14	1	0	1

Entropy(Play,ID) = 0
Gain(Play,ID) = **0.94**

Split (Play,ID) = - (1/14*$\log_2$(1/14)*14=3.81
Gain Ratio (Play,ID) = 0.94/3.81= **0.247**

The information gain for ID is high (0.94) without using the split information. However, with the adjustment the information gain dropped to 0.25.

ATTRIBUTES WITH DIFFERENT COSTS

- Sometimes the best attribute for splitting the training elements is very costly. In order to make the overall decision process more cost effective we may wish to penalize the information gain of an attribute by its cost.

$$G'(T,X) = \frac{G(T,X)}{Cost(X)}$$

Decision Tree - Regression

DECISION TREE

A set of training examples is broken down into smaller and smaller subsets while at the same time an associated decision tree is incrementally developed. At the end of the learning process, a decision tree covering the training set is returned.

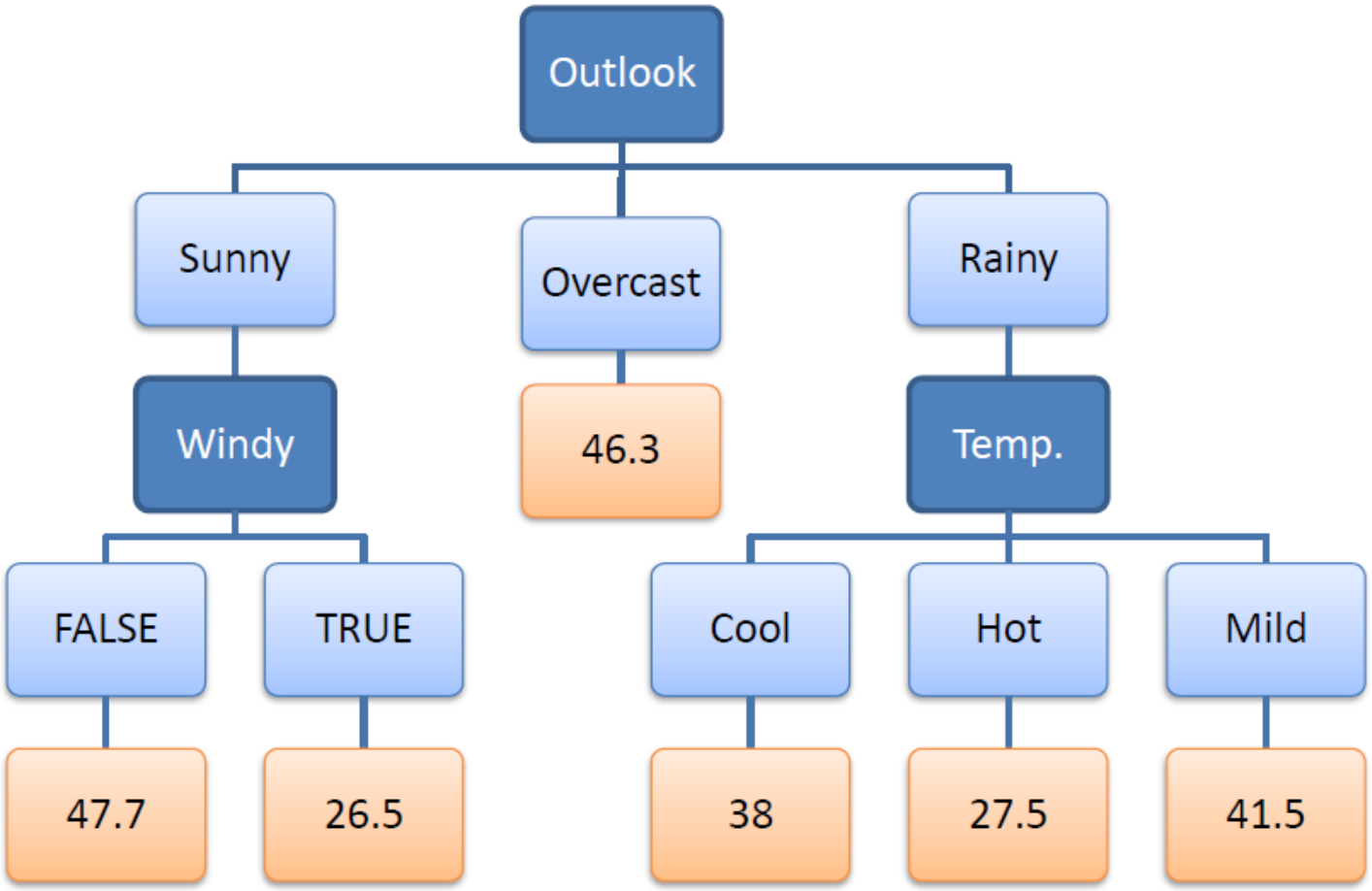
Mitchell, 1997

DATASET

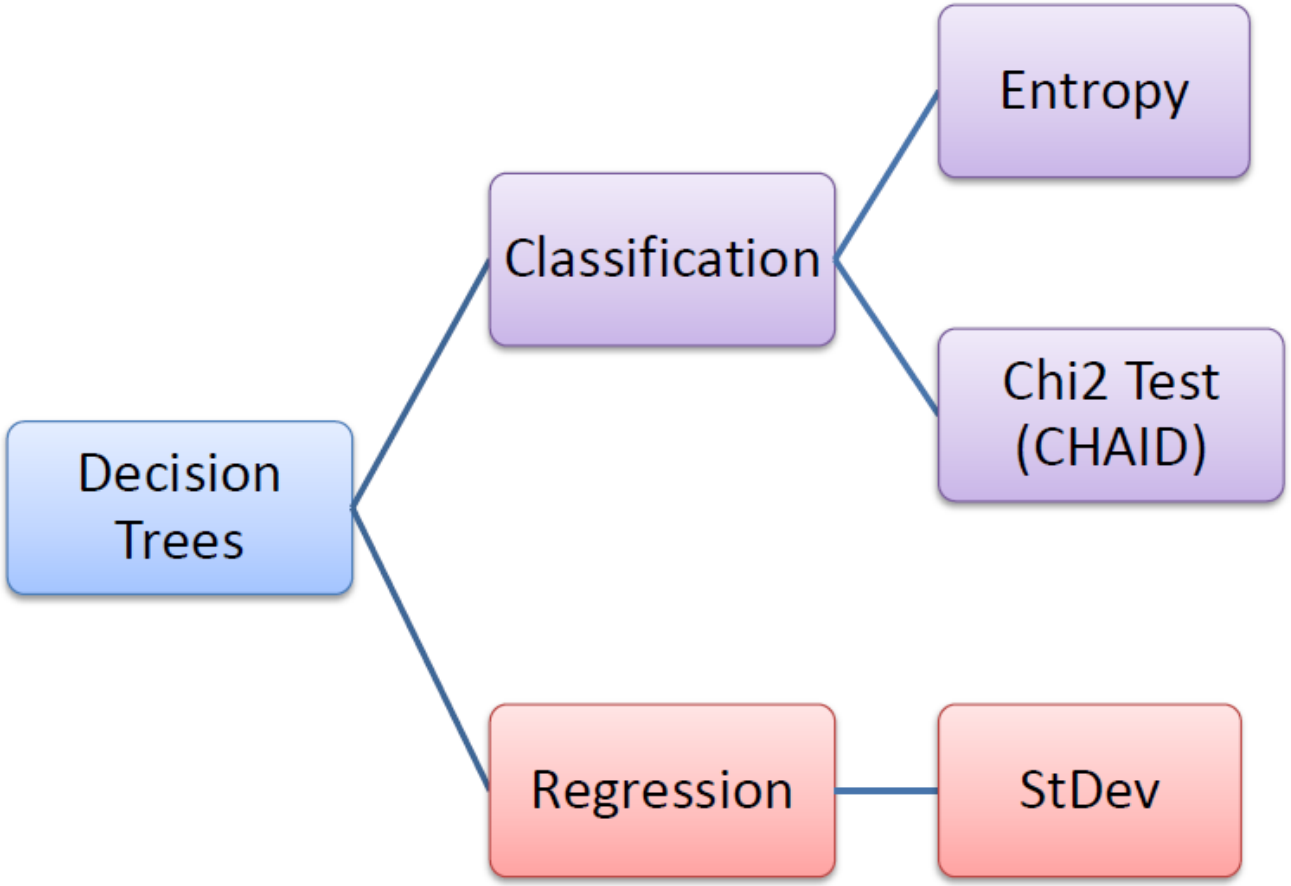
Predictors | Target

Outlook	Temp.	Humidity	Windy	Golf Players
Rainy	Hot	High	False	25
Rainy	Hot	High	True	30
Overcast	Hot	High	False	46
Sunny	Mild	High	False	45
Sunny	Cool	Normal	False	52
Sunny	Cool	Normal	True	23
Overcast	Cool	Normal	True	43
Rainy	Mild	High	False	35
Rainy	Cool	Normal	False	38
Sunny	Mild	Normal	False	46
Rainy	Mild	Normal	True	48
Overcast	Mild	High	True	52
Overcast	Hot	Normal	False	44
Sunny	Mild	High	True	30

DECISION TREE -REGRESSION



ENTROPY VERSUS STANDARD DEVIATION



$$E = \sum_{i=1}^{c} - p_i \log_2 p_i$$

$$\chi^2 = \sum_{i=1}^{r} \sum_{j=1}^{c} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

$$S = \sqrt{\frac{\sum (x - \mu)^2}{n}}$$

TARGET –STANDARD DEVIATION & AVERAGE

Golf Players
25
30
46
45
52
23
43
35
38
46
48
52
44
30

StDev = 9.32

Avg = 39.79

STANDARD DEVIATION TABLES

		Golf Players (StDev)
Outlook	Overcast	**3.49**
	Rainy	**7.78**
	Sunny	**10.87**

		Golf Players (StDev)
Temp.	Cool	**10.51**
	Hot	**8.95**
	Mild	**7.65**

		Golf Players (StDev)
Humidity	High	**9.36**
	Normal	**8.37**

		Golf Players (StDev)
Windy	False	**7.87**
	True	**10.59**

STANDARD DEVIATION

		Golf Players (StDev)	Count
Outlook	Overcast	**3.49**	**4**
	Rainy	**7.78**	**5**
	Sunny	**10.87**	**5**
			14

$$S(T,X) = \sum_{c \in X} P(c)S(c)$$

S(Players, Outlook) = **P**(Sunny)***S**(Sunny) + **P**(Overcast)***S**(Overcast) + **P**(Rainy)***S**(Rainy)

= (4/14)*3.49 + (5/14)*7.78 + (5/14)*10.87

= 7.66

STANDARD DEVIATION REDUCTION (SDR)

$$SDR(T,X) = S(T) - S(T,X)$$

G(Players, Outlook) = **S**(Players) – **S**(Players, Outlook)

= 9.32 – 7.66 = 1.66

STANDARD DEVIATION REDUCTION THE BEST PREDICTOR?

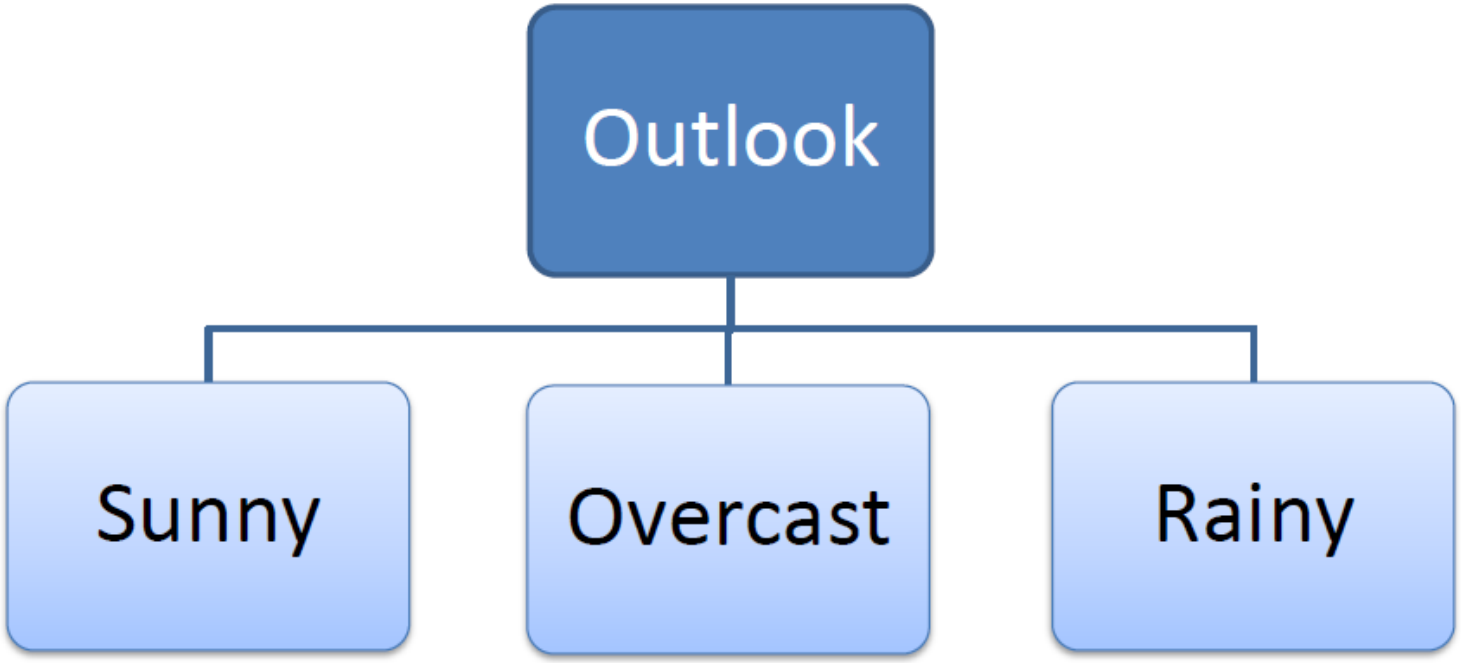
		Golf Players (StDev)
Outlook	Overcast	**3.49**
	Rainy	**7.78**
	Sunny	**10.87**
SDR=1.66		

		Golf Players (StDev)
Temp.	Cool	**10.51**
	Hot	**8.95**
	Mild	**7.65**
SDR=0.17		

		Golf Players (StDev)
Humidity	High	**9.36**
	Normal	**8.37**
SDR=0.28		

		Golf Players (StDev)
Windy	False	**7.87**
	True	**10.59**
SDR=0.29		

DECISION TREE –ROOT NODE



DATASET –SORTED BY OUTLOOK

Outlook	Temp.	Humidity	Windy	Golf Players
Sunny	Mild	High	FALSE	45
Sunny	Cool	Normal	FALSE	52
Sunny	Cool	Normal	TRUE	23
Sunny	Mild	Normal	FALSE	46
Sunny	Mild	High	TRUE	30
Rainy	Hot	High	FALSE	25
Rainy	Hot	High	TRUE	30
Rainy	Mild	High	FALSE	35
Rainy	Cool	Normal	FALSE	38
Rainy	Mild	Normal	TRUE	48
Overcast	Hot	High	FALSE	46
Overcast	Cool	Normal	TRUE	43
Overcast	Mild	High	TRUE	52
Overcast	Hot	Normal	FALSE	44

SUBSET (OUTLOOK = SUNNY)

Temp.	Humidity	Windy	Golf Players
Mild	High	FALSE	45
Cool	Normal	FALSE	52
Cool	Normal	TRUE	23
Mild	Normal	FALSE	46
Mild	High	TRUE	30
			SD=10.87

		Golf Players (StDev)
Temp.	Cool	14.50
	Mild	7.32
SDR= 0.678		

		Golf Players (StDev)
Humidity	High	7.50
	Normal	12.50
SDR= 0.370		

		Golf Players (StDev)
Windy	False	3.09
	True	3.50
SDR= 7.62		

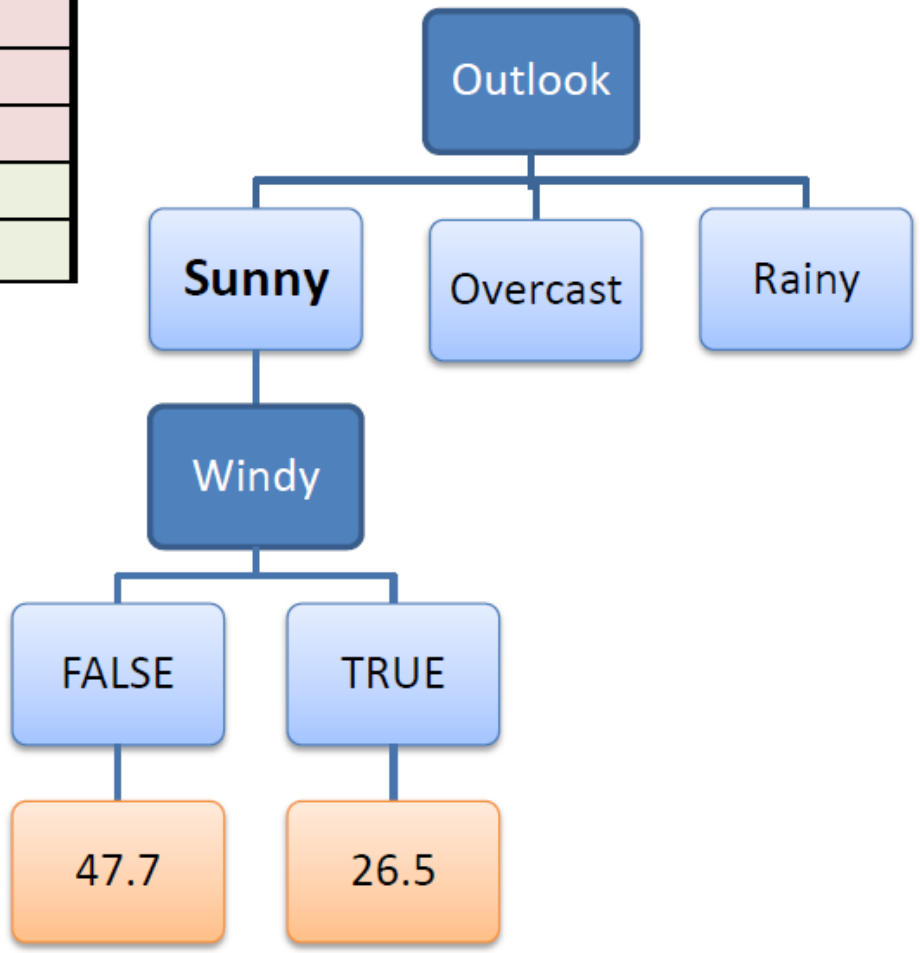
SDR = 10.87-((2/5)*14.5 + (3/5)*7.32)

SDR = 10.87-((2/5)*7.5 + (3/5)*12.5)

SDR = 10.87-((3/5)*3.09 + (2/5)*3.5)

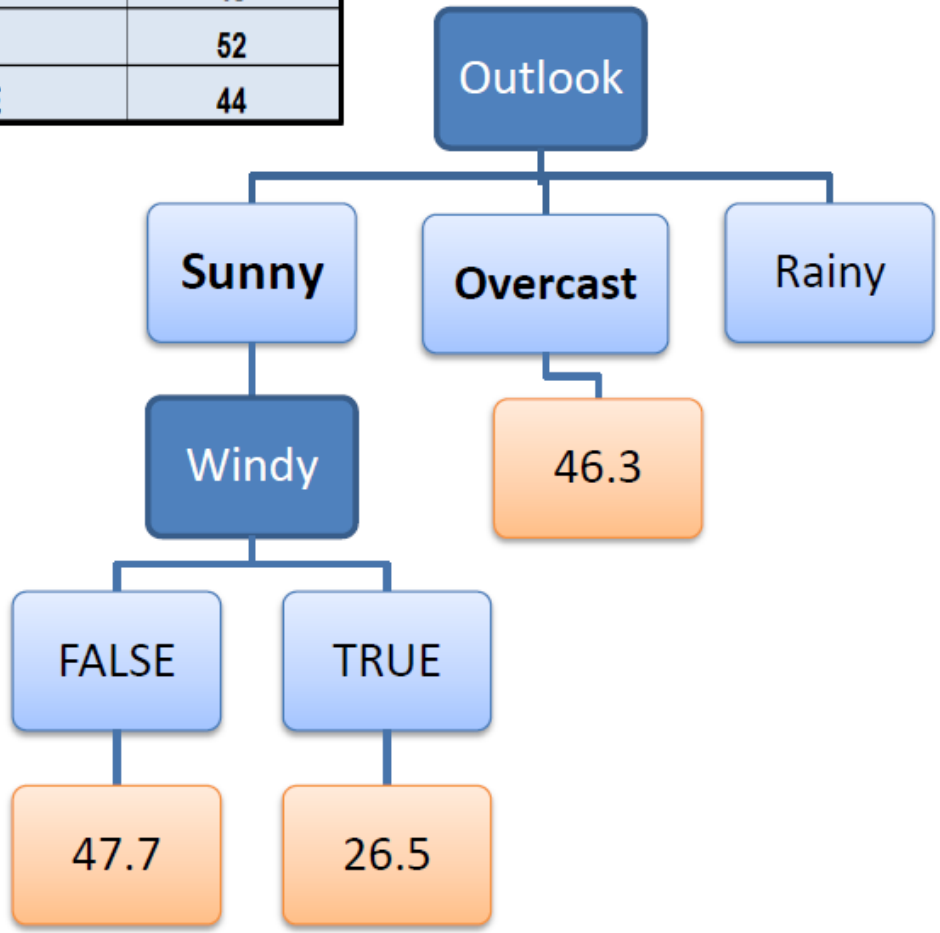
SUBSET (OUTLOOK = SUNNY)

Temp.	Humidity	Windy	Golf Players
Mild	High	FALSE	45
Cool	Normal	FALSE	52
Mild	Normal	FALSE	46
Cool	Normal	TRUE	23
Mild	High	TRUE	30



SUBSET (OUTLOOK = OVERCAST)

Overcast	Hot	High	FALSE	46
Overcast	Cool	Normal	TRUE	43
Overcast	Mild	High	TRUE	52
Overcast	Hot	Normal	FALSE	44



SUBSET (OUTLOOK = RAINY)

Temp.	Humidity	Windy	Golf Players
Hot	High	FALSE	25
Hot	High	TRUE	30
Mild	High	FALSE	35
Cool	Normal	FALSE	38
Mild	Normal	TRUE	48
			StDev=7.78

★		Golf Players (StDev)
Temp.	Cool	0
	Hot	2.5
	Mild	6.5
SDR= 4.18		

SDR = 7.78 - ((2/5)*2.5 + (2/5)*6.5)

		Golf Players (StDev)
Humidity	High	4.1
	Normal	5.0
SDR= 3.32		

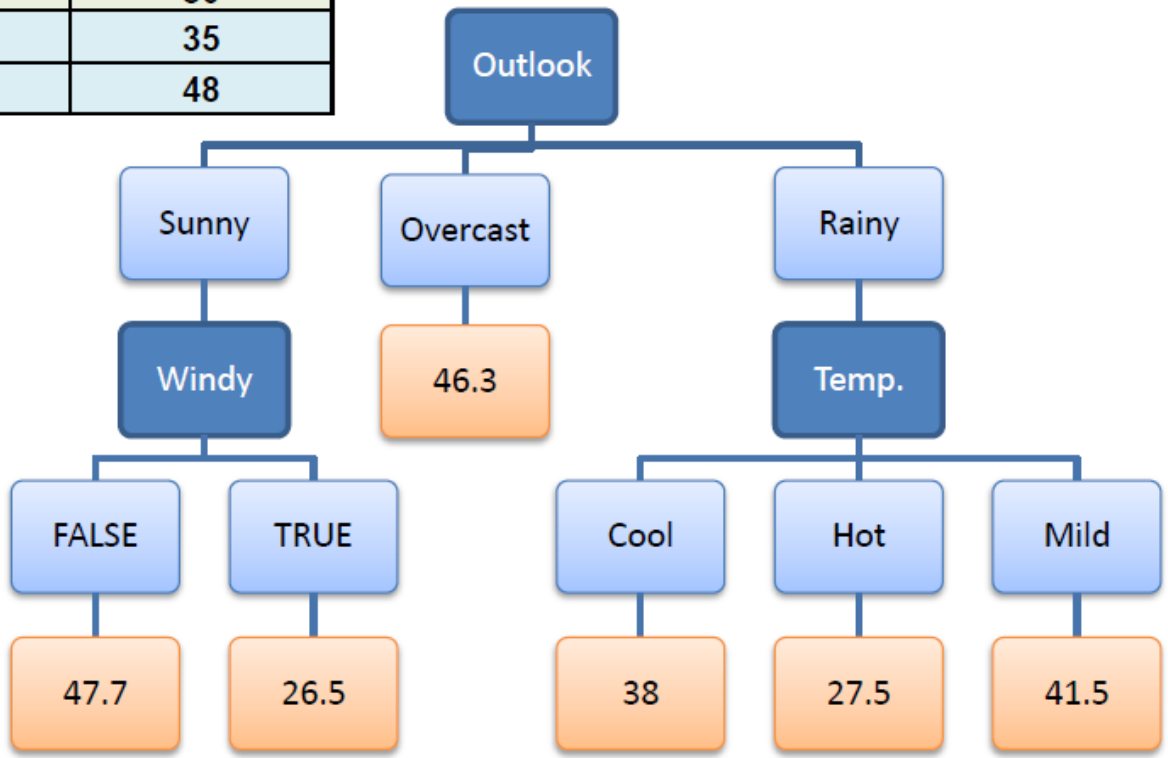
SDR = 7.78 - ((3/5)*4.1 + (2/5)*5.0)

		Golf Players (StDev)
Windy	False	5.6
	True	9.0
SDR= 0.82		

SDR = 7.78 - ((3/5)*5.6 + (2/5)*9.0)

SUBSET (OUTLOOK = RAINY)

Temp.	Humidity	Windy	Golf Players
Cool	Normal	FALSE	38
Hot	High	FALSE	25
Hot	High	TRUE	30
Mild	High	FALSE	35
Mild	Normal	TRUE	48



ADVANTAGES OF THE DECISION TREE

Classification using a decision tree is extremely fast and it is simple to understand as it follows the same process which a human follow while making any decision in real-life.

It can be very useful for solving decision-related problems.

It helps to think about all the possible outcomes for a problem.

There is less requirement of data cleaning compared to other algorithms.

Data versatility: Decision trees can handle both numerical and categorical data.

Feature selection: The process naturally identifies and ranks the most important features for making a prediction, which aids in feature selection.

Robustness: They are less sensitive to outliers and noisy data compared to some other algorithms.

Versatility: They are used in various applications, including classification, regression, and as a component in more complex ensemble methods like Random Forest.

DISADVANTAGES OF THE DECISION TREE

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm.**
- For more class labels, the computational complexity of the decision tree may increase.
- Post-pruning is more effective but involves more computational cost. It is based on the evidence of the result of a complete tree

METRICS FOR MODEL EVALUATION

The Confusion Matrix evaluates the ability of a classifier based on the following indicators

- TP (true positive): records correctly classified as Yes class
- FN (false negative): Incorrectly classified records as class No
- FP (false positive): Incorrectly classified records as class Yes
- TN (true negative) records correctly classified as class No

		Expected Class	
		Class=Yes	Class=No
Actual Class	Class=Yes	TP	FN
	Class=No	FP	TN

If the classification uses n classes, the confusion matrix will be $n \times n$

ACCURACY

Accuracy is the most widely used metric to synthesize the information of a confusion matrix

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Equally, the frequency of the error could be used

$$\text{Error rate} = \frac{FP + FN}{TP + TN + FP + FN}$$

ACCURACY LIMITATIONS

Accuracy is not an appropriate metric if the classes contain a very different number of records. Consider a binary classification problem in which

- # Record of class 0 = 9990
- # Record of class 1 = 10

A model that always returns class 0 will have an accuracy of 9990/10000 = 99.9%

In the case of binary classification problems, the class "rare" is also called a positive class, while the class that includes most of the records is called a negative class

PRECISION AND RECALL

Precision and Recall are two metrics used in applications where <u>the correct classification of positive class records is more important</u>.

Precision is the ratio of true positives to all positive predictions.

- High values indicate that few negative class records were incorrectly classified as positive.

Recall is the ratio of true positives to all actual positive cases.

- High values indicate that few records of the positive class were incorrectly classified as negatives.

$$\text{Precision, } p = \frac{TP}{TP + FP}$$

$$\text{Recall, } r = \frac{TP}{TP + FN}$$

PRECISION AND RECALL

Precision = 1 if all the positive records were actually detected.

Recall = 1 if there are no false negatives

EXAMPLE

- A model makes predictions and predicts 120 examples as belonging to the minority class, 90 of which are correct, and 30of which are incorrect.
- The precision for this model is calculated as:
- Precision=True Positives / (True Positives + False Positives)
- Precision = 90 / (90 + 30)
- Precision = 90 / 120
- Precision = 0.75

F-MEASURE

A metric that summarizes precision and recall is called F-measure or F1-Score

This score takes false positives and false negatives into account.

$$\text{F-measure, F} = \frac{2rp}{r+p} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

F-measure represents the harmonic mean of precision and recall

If the F-measure is high, it means both precision and recall are.

F-measure is usually more useful than accuracy, especially if you have uneven class distribution.

Consider the confusion matrix given below for a binary classifier predicting the presence of a disease

The classifier made a total of 150 predictions

Out of those 150 cases, the classifier predicted "yes" 100 times, and "no" 50 times.

In reality, 100 patients in the sample have the disease, and 50 patients do not.

	Predicted No	Predicted Yes
Actual No	TN = 45	FP = 5
Actual Yes	FN = 5	TP = 95

$$Precision = \frac{TP}{Predicted\ Yes} = \frac{95}{100} = 95\%$$

$$Accuraccy = \frac{TN+TP}{TN+FP+FN+TP} = \frac{45+95}{150} = 93.33\%$$

F1 score: 66.67%

CLASSIFICATION ERRORS

Training error: are mistakes that are made on the training set

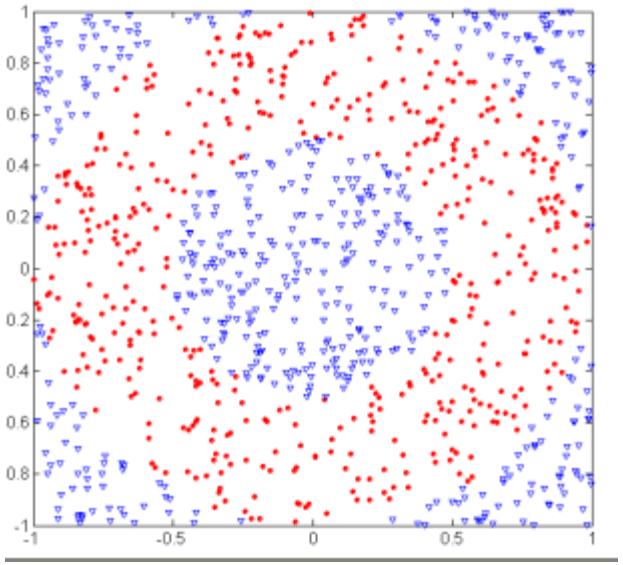
Generalization error: errors are made on the test set (i.e. records that have not been trained on the system).

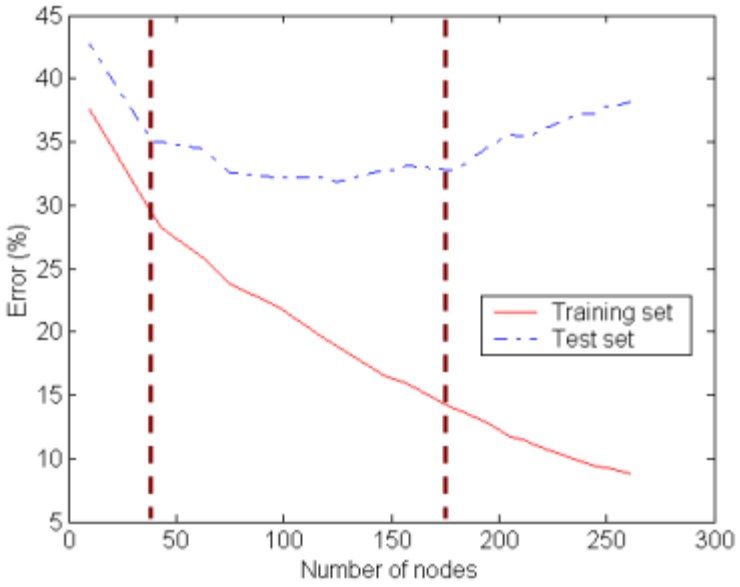
Underfitting: The model is too simple and does not allow a good classification or set training set or test set

Overfitting: The model is too complex, it allows a good classification of the training set, but a poor classification of the test set

- The model fails to generalize because it is based on the specific peculiarities of the training set that are not found in the test set (e.g. noise present in the training set)

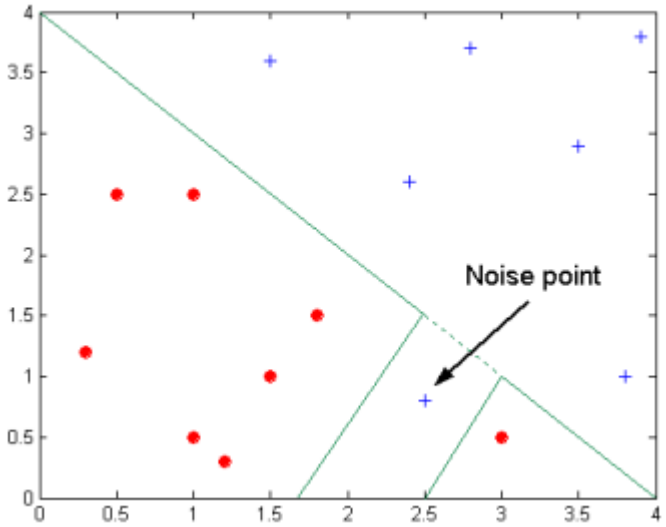
UNDERFITTING AND OVERFITTING



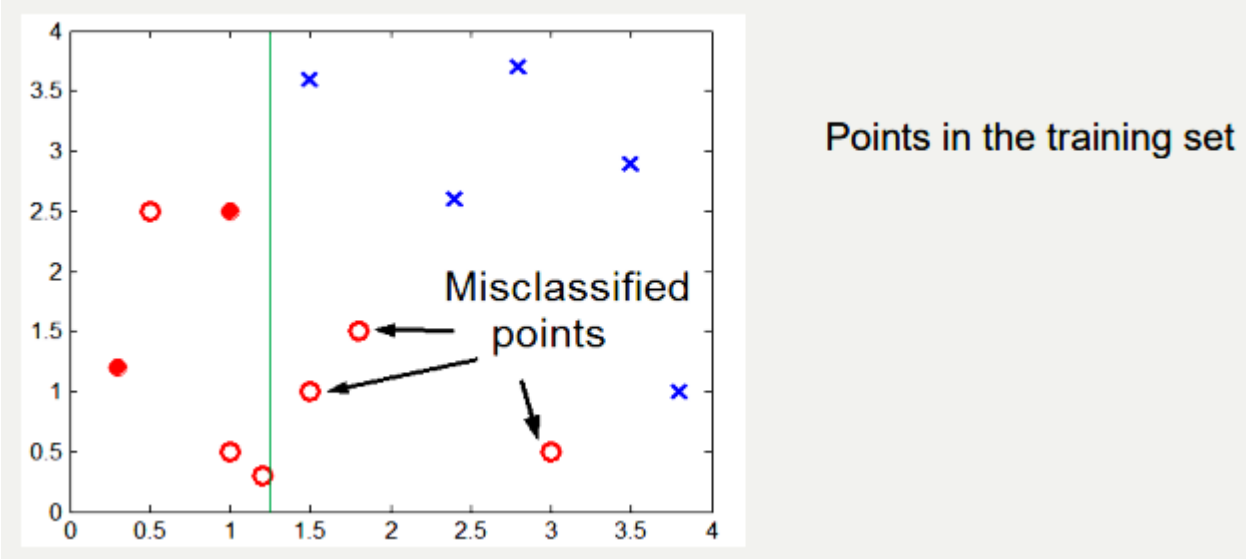


OVERFITTING DUE TO NOISE

The boundaries of the areas are distorted due to noise.



OVERFITTING DUE TO THE REDUCED SIZE OF THE TRAINING SET



THANK YOU